

Transport Layer: UDP & TCP

1. Process-to-Process Delivery

Recall the OSI/TCP-IP layering: Data Link does **node-to-node** delivery, Network does **host-to-host** delivery. But real communication happens between **specific processes** (e.g., a browser process on your laptop talking to a web server process on a remote machine) — several processes can be running on both ends simultaneously, so host-to-host delivery alone isn't enough. The **Transport layer** closes this last gap: **process-to-process delivery**.

- **Addressing at each layer:** MAC address (Data Link, picks one node) → IP address (Network, picks one host among millions) → **port number** (Transport, picks one process among the many running on that host).
- **Socket address** = IP address + port number — uniquely identifies one endpoint of a connection. A full connection needs a socket address at **both** ends (client socket address + server socket address) — this pair of socket addresses is what actually identifies a specific communication session.

Port ranges (IANA)

Range	Name	Usage
0 – 1023	Well-known	Assigned/controlled by IANA (e.g., 80=HTTP, 443=HTTPS, 53=DNS)
1024 – 49,151	Registered	Not IANA-controlled, but can be registered to avoid clashes
49,152 – 65,535	Dynamic / Ephemeral	Freely used by any process — typically the client's temporary source port for an outgoing connection

2. Connectionless vs Connection-Oriented Service

- **Connectionless** — packets sent with no setup/teardown; not numbered, may be lost/duplicated/reordered, no acknowledgment. **UDP** is connectionless.
 - **Connection-oriented** — a connection is explicitly established first, data is transferred, then the connection is explicitly released. **TCP** (and SCTP) are connection-oriented.
-

3. UDP (User Datagram Protocol)

Header (8 bytes, fixed)

Source Port (16 bits)	Destination Port (16 bits)
Total Length (16 bits)	Checksum (16 bits)

- **Length field is technically redundant** — the IP datagram carrying this UDP segment already has a total-length field and a header-length field; subtracting one from the other yields the UDP length anyway. It's kept mainly for convenience/clarity.
- **Checksum is optional** in UDP (unlike TCP, where it's mandatory) — reflecting UDP's whole philosophy of minimal overhead.

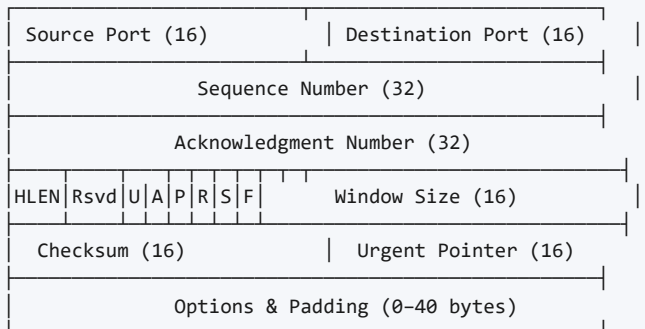
Why UDP exists, despite offering no reliability

- Suitable for simple **request-response** communication with minimal flow/error-control needs.
- Suitable when the **application itself** already implements its own flow/error control (e.g., TFTP).
- **The only transport protocol supporting multicasting** — TCP's connection-oriented, per-pair-acknowledgment model is fundamentally incompatible with one-to-many delivery.
- Used for management/lightweight protocols where speed matters more than guaranteed delivery: **SNMP**, **RIP** route updates, DNS queries, and modern real-time media (VoIP, video streaming) where a late-arriving retransmission is *worse* than just dropping the lost data and moving on.

Interview soundbite: "UDP isn't 'TCP without reliability' as a compromise — it's a deliberate choice for exactly the situations where TCP's guarantees are either unnecessary (the app has its own error handling), actively counterproductive (real-time media, where a retransmitted-but-late packet is useless), or structurally impossible for TCP to provide at all (multicast)."

4. TCP (Transmission Control Protocol)

Header (20–60 bytes)

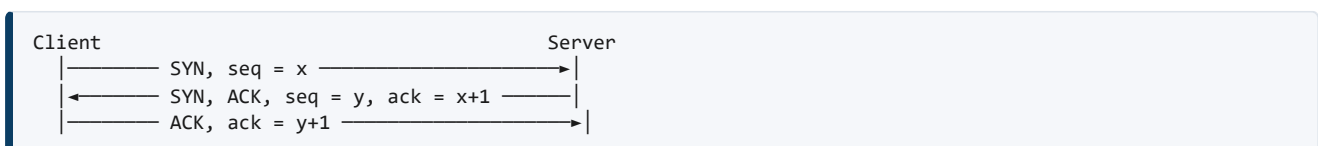


- **Sequence number** — the number of the **first data byte** in this segment. TCP numbers every individual **byte** of the stream (not the segment), which is what "TCP is a byte-stream protocol" actually means in practice.
- **Acknowledgment number** — the **next byte** the receiver expects. If the receiver has successfully gotten byte **x**, it sends back acknowledgment number **x+1**. Ack and data can be **piggybacked** together in the same segment (same idea as in the Data Link ARQ topic).
- **HLEN** — header length in 4-byte words (same convention as the IP header).
- **Window size** — how many bytes the *other* side is currently willing to receive (the **receive window, rwnd**) — the sender must never have more unacknowledged bytes in flight than this.
- **Checksum** — mandatory in TCP (unlike UDP's optional checksum).

The 6 control flags

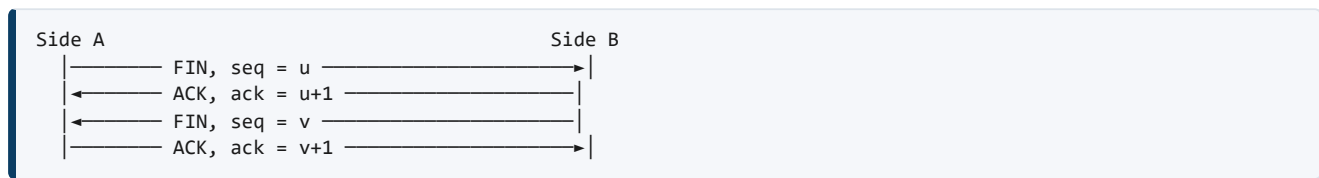
Flag	Meaning
SYN	Synchronize sequence numbers — sent only in connection establishment
ACK	The acknowledgment number field is valid
FIN	Sender has no more data — request graceful connection termination
RST	Abruptly reset/refuse a connection (something is wrong, or an unexpected packet arrived)
PSH	Push buffered data to the application layer immediately, instead of waiting to accumulate more (important for interactive apps like chat, where waiting for a full segment's worth of data would add unacceptable delay)
URG	Segment contains urgent data that should be processed by the receiver ahead of normal buffered data; the urgent pointer field marks where the urgent data ends

5. Connection Establishment — the 3-Way Handshake



1. **Client → Server: SYN.** Client picks a random **Initial Sequence Number (ISN)**, `x`. No data, no ack number — but it consumes 1 sequence number, since it needs to be acknowledged.
 2. **Server → Client: SYN + ACK.** This segment does double duty — it's a SYN for the server-to-client direction (server picks its own random ISN, `y`) *and* an ACK of the client's SYN (`ack = x+1`). Since it carries an ACK, it must also specify the server's receive window (rwnd).
 3. **Client → Server: ACK.** Simple acknowledgment (`ack = y+1`); if it carries no data, it **doesn't consume a sequence number**.
- **Active open** — whoever initiates the connection (the client). **Passive open** — whoever is listening for it (the server).

6. Connection Termination — 4-Way (graceful close)



- A FIN (like a SYN) consumes one sequence number if it carries no data.
- Termination is **independent per direction** — this is exactly why it takes 4 messages, not 3: side A saying "I'm done sending" doesn't mean side B is also done sending its own data, so each direction gets its own FIN/ACK pair.

7. Flow Control

- The receiver advertises how much buffer space it currently has via the **Window Size (rwnd)** field.
- **The sender must obey this** — it can never have more unacknowledged data in flight than the receiver's currently advertised window, preventing a fast sender from overwhelming a slow receiver's buffer.

8. Fast Retransmit

TCP normally waits for a **retransmission timeout (RTO)** before assuming a segment was lost. Fast retransmit is a faster, complementary mechanism:

- If a segment arrives **out of order** (because an earlier one was lost/delayed), the receiver re-sends the **same** acknowledgment number it sent last time (it can't ACK the new, out-of-order data, since there's still a gap before it).
- These repeated ACKs are called **duplicate ACKs**.
- When the sender sees **3 duplicate ACKs**, it concludes a segment was almost certainly lost — and **retransmits it immediately**, without waiting for the RTO timer to expire at all.

Why 3, not 1? A single duplicate ACK could just mean packets arrived slightly out of order (common, harmless) rather than a genuine loss. Requiring 3 duplicates before reacting filters out this normal reordering noise while still reacting much faster than waiting for a full timeout.

9. Congestion Control

The core mechanism: the congestion window (cwnd)

TCP maintains a second window — the **congestion window (cwnd)** — reflecting how much data the **network** (not just the receiver) can currently handle without becoming congested. The sender's actual allowed data in flight is $\min(\text{cwnd}, \text{rwnd})$ — flow control (rwnd) and congestion control (cwnd) are two independent constraints, both must be respected simultaneously.

Slow Start

- Starts with a small **cwnd** (historically 1 MSS).
- **Doubles cwnd every RTT** (exponential growth) — despite the name "slow start," the growth rate is actually aggressive; the name refers only to starting from a small initial value, not to how fast it grows afterward.
- Continues until **cwnd** reaches the **slow start threshold (sssthresh)** — at which point it transitions to congestion avoidance.

Congestion Avoidance — AIMD (Additive Increase, Multiplicative Decrease)

- Once $\text{cwnd} \geq \text{sssthresh}$: **additive increase** — **cwnd** grows by roughly 1 MSS per RTT (linear growth, much more cautious than slow start's doubling).
- **Multiplicative decrease** — on detecting congestion, **sssthresh** is set to **half** the current **cwnd** (not **cwnd** itself halved directly — **sssthresh** is what's halved).

TCP Tahoe — the original scheme

Slow Start + AIMD + Fast Retransmit. On **any** sign of congestion (timeout **or** 3 duplicate ACKs), Tahoe reacts the same aggressive way: $\text{sssthresh} = \text{cwnd}/2$, and **cwnd** is reset all the way down to **1 MSS**, restarting slow start from scratch.

Worked example: **cwnd = 200**, a packet loss occurs.

```
sssthresh = cwnd / 2 = 100
cwnd reset to 1
→ Slow start begins again, doubling each RTT, until cwnd reaches sssthresh (100)
→ Then congestion avoidance (AIMD) takes over, growing cwnd by ~1 per RTT
```

If a **further** loss occurs later while **cwnd** had grown to, say, 125 under AIMD:

```
sssthresh = 125 / 2 ≈ 62
cwnd reset to 1 again → slow start resumes toward the new sssthresh (62)
```

TCP Reno — adds Fast Recovery

TCP Reno = TCP Tahoe + Fast Recovery. The key improvement: Reno treats the **two** possible congestion signals *differently*, rather than reacting identically to both like Tahoe does.

- **On timeout (RTO)** — treated as a sign of *serious* congestion (packets are being dropped and nothing is getting through at all): behaves like Tahoe — $ssthresh = cwnd/2$, $cwnd$ reset to 1, restart slow start.
- **On 3 duplicate ACKs (fast retransmit)** — treated as a *milder* signal, since duplicate ACKs mean packets *are* still getting through and being acknowledged (the network isn't fully collapsed, just moderately congested): enters **Fast Recovery** instead of collapsing all the way to slow start.
 $ssthresh = cwnd / 2$ $cwnd = ssthresh + 3 \times MSS$ (not reset to 1!)

This lets $cwnd$ resume growth from roughly *half* its prior value (behaving like AIMD immediately) rather than being forced to re-climb all the way from 1 via slow start — since the 3-duplicate-ACK signal already indicates the network is only moderately, not catastrophically, congested.

Interview soundbite: "Tahoe treats every loss signal the same — nuke $cwnd$ to 1 and start over. Reno's key insight is that 3 duplicate ACKs and a timeout mean fundamentally different things: dup-ACKs mean segments are still flowing and being acknowledged, just with one gap — so Reno only *halves* things and jumps back into linear growth (Fast Recovery), reserving the full reset-to-1 punishment for when a timeout proves the network has gone properly silent."

Summary

Event	Tahoe	Reno
3 duplicate ACKs	$ssthresh=cwnd/2$, $cwnd=1$, restart slow start	$ssthresh=cwnd/2$, $cwnd=ssthresh+3MSS$, enter Fast Recovery ($\approx$ AIMD)
Timeout (RTO)	$ssthresh=cwnd/2$, $cwnd=1$, restart slow start	$ssthresh=cwnd/2$, $cwnd=1$, restart slow start

Interview Questions With Answers

Q1. Why is a socket address (IP + port) necessary instead of just a port number to identify a process?

Answer: A port number alone only identifies *which process* on a given machine, but doesn't say *which machine* — the same port number (e.g., 443) is simultaneously in use on millions of different servers worldwide. Combining the IP address (identifying the specific host) with the port number (identifying the specific process on that host) is what uniquely identifies one actual communication endpoint anywhere on the network.

Q2. Why is UDP's checksum optional while TCP's is mandatory?

Answer: This reflects the two protocols' different design philosophies: UDP prioritizes minimal overhead and speed for applications that don't need strong guarantees (and which may implement their own error handling), so even the modest cost of mandatory checksumming is left as the application's choice. TCP is built around guaranteeing reliable delivery as a core promise, so a mandatory checksum on every segment is a non-negotiable part of that reliability contract — silently accepting corrupted data would violate TCP's fundamental purpose.

Q3. Walk through the 3-way handshake and explain why the second step needs to be both a SYN and an ACK combined.

Answer: The client sends a SYN with its chosen initial sequence number. The server needs to do two things in response: acknowledge the client's SYN (so the client knows the server received it) *and* establish its own initial sequence number for the server-to-client direction of the connection (since TCP connections are full-duplex, with independent sequence numbering in each direction) — combining these into one SYN+ACK segment avoids needing two separate round trips for what's logically one "the server is also ready" event. The client then completes the handshake with a final ACK of the server's SYN.

Q4. Why does TCP connection termination require 4 messages instead of 3, unlike establishment?

Answer: Because TCP is full-duplex and each direction of the connection can be closed independently — one side sending a FIN only signals "I have no more data to send," not that the other side is also finished. So side A's FIN is acknowledged separately from side B's own FIN (sent whenever B is actually done), giving 4 total messages (FIN, ACK, FIN, ACK) rather than combining a FIN+ACK the way SYN+ACK is combined in the handshake, since B may still have data left to send after receiving A's FIN and isn't necessarily ready to send its own FIN at the same moment.

Q5. Why does fast retransmit wait for 3 duplicate ACKs rather than reacting to the very first one?

Answer: A single duplicate ACK can occur simply because segments arrived slightly out of order due to normal network variability (different routing paths, minor jitter) rather than an actual loss — reacting immediately to just one would cause unnecessary retransmissions for what might just be reordering. Requiring 3 duplicates before triggering retransmission filters out this common, harmless reordering noise, while still reacting far faster than waiting for the full retransmission timeout, which is the whole point of the mechanism.

Q6. What's the difference between flow control (rwnd) and congestion control (cwnd), and why does TCP need both simultaneously?

Answer: Flow control (rwnd) protects the *receiver* — it reflects how much buffer space the receiving application currently has, preventing a fast sender from overwhelming a slow receiver. Congestion

control (cwnd) protects the *network* itself — it reflects how much data the sender estimates the network's intermediate links/routers can currently handle without becoming congested, independent of whether the receiver's buffer has room. TCP needs both because these are genuinely different bottlenecks that can each independently limit safe throughput — a receiver could have plenty of buffer space while the network path is congested, or vice versa — so the sender always respects whichever constraint ($\min(\text{cwnd}, \text{rwnd})$) is currently smaller.

Q7. Why is "slow start" actually an aggressive, exponentially-growing phase, despite its name?

Answer: The name refers only to the *starting point* — cwnd begins very small (historically 1 MSS) rather than jumping immediately to a large value, which is the "slow" part. But once underway, cwnd doubles every round-trip time, which is exponential growth — genuinely fast, not slow, in terms of *rate* of increase. The name describes caution about the *initial* value, not caution about the growth *rate* that follows it.

Q8. Why does TCP Reno treat a timeout and 3 duplicate ACKs differently, while Tahoe treats them the same?

Answer: A timeout means no acknowledgment at all has arrived for a segment within the expected time — a strong signal that packets may be getting dropped wholesale and the network path may be seriously congested or broken. Three duplicate ACKs, by contrast, mean packets *are* still successfully reaching the receiver and being acknowledged — just with one specific gap — which is evidence of only moderate congestion, not a collapsed path. Reno's Fast Recovery mechanism exploits this distinction: since duplicate ACKs indicate the network is still largely functioning, Reno only halves cwnd (via $\text{ssthresh} = \text{cwnd} / 2$, $\text{cwnd} = \text{ssthresh} + 3\text{MSS}$) instead of nuking it all the way to 1 and re-running the full slow start climb, recovering throughput much faster after a moderate congestion event. Tahoe predates this insight and reacts identically (full reset to 1) to both signals, treating every loss as equally severe.

Q9. Scenario: A TCP connection's cwnd is 64 (already past ssthresh, in congestion avoidance) when it experiences 3 duplicate ACKs, and shortly afterward — before fully recovering — a timeout also occurs. Trace what happens to cwnd and ssthresh under TCP Reno through both events.

Answer: At the 3-duplicate-ACK event: $\text{ssthresh} = 64 / 2 = 32$, and Reno enters Fast Recovery with $\text{cwnd} = \text{ssthresh} + 3 \times \text{MSS} = 32 + 3 = 35$ (in MSS units), rather than collapsing to 1 — the connection continues sending at roughly half its prior rate rather than restarting from scratch. If a **timeout** then occurs shortly after (before congestion avoidance has fully re-stabilized), Reno treats this as the more severe signal regardless of the recent Fast Recovery: $\text{ssthresh} = \text{current cwnd} / 2$ (roughly $35 / 2 \approx 17$, using whatever cwnd had reached by that point), and **cwnd** is reset all the way down to **1 MSS**, forcing a full restart via slow start up to the new, much lower ssthresh (~17) before congestion avoidance resumes. This illustrates that Reno's leniency (Fast Recovery) is specifically reserved for

the dup-ACK signal — a subsequent timeout always triggers the full, severe reset regardless of what happened moments earlier.